

RESULTS

Exam Results

653 / 1000

NOT PASSED

39 of 60 correct (65%) · Pass mark: 720

Domain Breakdown

D1

Agentic Architecture & Orchestration

11/14 (79%) +212pts

D2

Tool Design & MCP Integration

4/11 (36%) +65pts

D3

Claude Code Configuration & Workflows

10/12 (83%) +167pts

D4

Prompt Engineering & Structured Output

6/12 (50%) +100pts

D5

Context Management & Reliability

8/11 (73%) +109pts

HOW SCORING WORKS

Each domain's percentage correct is multiplied by its weight and scaled to 1000 points. For example, scoring 80% in Agentic Architecture (27% weight) contributes $0.80 \times 0.27 \times 1000 = 216$ points. The pass mark is 720/1000 (72%). If a domain has no questions in the selected scenarios, its weight is redistributed proportionally among assessed domains.

The real exam reports a scaled score from 100 to 1,000 plus your percent-correct by domain, so treat the percentages here as a readiness gauge rather than a prediction of your scaled score.

HIDE DETAIL

TAKE ANOTHER EXAM

Question-by-Question Review

ALL 60

CORRECT 39

INCORRECT 21

SKIPPED 0

Q01 · D2 · 2.3 TOOL-DISTRIBUTION-CHOICE / ROLE-SCOPING · q-2-3-002 · Developer Productivity Tools

INCORRECT

A developer productivity platform has a code review agent equipped with 18 tools covering code analysis, documentation lookup, test generation, dependency checking, security scanning, and deployment validation. Reviews are slow and the agent frequently selects the wrong tool. What is the most effective architectural change?

C

Switch tool_choice from 'auto' to 'any' so the model is forced to select a tool on every turn.

Forcing tool selection does not help when the agent cannot reliably choose among 18 options. The problem is selection accuracy due to overload, not whether the model calls a tool at all.

D Split the agent into role-specific agents with 4-5 tools each, scoped to their specialisation.

The recommended range is 4-5 tools per agent. Splitting into role-specific agents (for example separate agents for code analysis, security scanning, and test generation) eliminates decision complexity and prevents cross-specialisation misuse. Each agent excels within its narrow remit. **(correct answer)**

B Implement a routing classifier that pre-selects the appropriate tool before each agent turn.

A routing classifier adds infrastructure complexity and bypasses the LLM's natural language understanding. The root cause is too many tools, not inadequate routing. **(your answer)**

A Rewrite all 18 tool descriptions with explicit boundaries and 'use this when' guidance so the agent can tell them apart.

Better descriptions help but do not solve the fundamental problem: 18 tools exceed the recommended range and create too much decision complexity. Selection accuracy degrades regardless of description quality when tool count is too high.

WHERE THIS COMES FROM

Lesson 2.3: Tool Distribution and Tool Choice (Role-specific scoping)

Lesson 2.3: Tool Distribution and Tool Choice (Tool overload)

Q02 · D5 5.5 HUMAN-REVIEW-CALIBRATION / AGGREGATE-METRICS-TRAP q-5-5-001 · Structured Data Extraction Pipeline **CORRECT**

A structured data extraction system achieves 97% overall accuracy across all document types. The team proposes automating all extractions where model confidence exceeds 95% to reduce human review costs. What is the critical risk in this approach?

D Automated extractions should always have human review regardless of confidence, making the proposal fundamentally flawed

Automation is valid when properly validated — the issue is doing it based on uncalibrated aggregate metrics, not the concept of automation itself.

C The model will become overconfident over time as it processes more documents, requiring regular retraining

LLMs do not train during inference. The issue is existing calibration gaps, not drift.

B Aggregate accuracy can mask poor per-type performance, and confidence scores need calibration before use.

97% overall can hide 40% error rates on specific document types. Without stratified validation and confidence calibration, automation will silently fail on certain inputs.

A The 95% confidence threshold is too low and should be raised to 99% for automation

The threshold value is not the core issue — the problem is that aggregate metrics hide per-type performance disparities.

WHERE THIS COMES FROM

Lesson 5.5: Human Review and Confidence Calibration (Aggregate metrics trap)

Lesson 5.5: Human Review and Confidence Calibration (Confidence calibration)

Q03 · D1 1.1 AGENTIC-LOOPS / MODEL-DRIVEN q-1-1-008 · Content Moderation and Classification System CORRECT

The moderation system's agentic loop uses a hardcoded decision tree: if the `classify_content` tool returns 'hate_speech', always call `escalate_to_human`; if it returns 'spam', always call `auto_remove`. During testing, the team discovers that satirical posts criticising hate speech are being auto-escalated, and sophisticated spam disguised as legitimate marketing slips through. What is the architectural problem?

B Replace the hardcoded decision tree with model-driven decisions, letting Claude weigh the full context of each post before it acts.

Hardcoded decision trees treat classification labels as absolute when real content is nuanced. Letting Claude reason about context (satire vs genuine hate, sophisticated spam patterns) produces better moderation decisions. The agentic loop should let the model decide, not map labels to fixed actions.

C Add a confidence threshold so only high-confidence classifications trigger automatic actions

Confidence thresholds reduce false positives but the fundamental problem remains: a hardcoded tree cannot handle contextual nuance. Low-confidence cases still need model-driven reasoning, not just deferral.

D Route all ambiguous cases to human review to avoid misclassification

Routing everything ambiguous to humans defeats the purpose of automated moderation. The model can reason about context effectively when given the opportunity; the hardcoded tree is what prevents it.

A The `classify_content` tool needs more granular category labels so it can tell satire criticising hate speech apart from genuine hate speech.

More granular labels do not address the core issue. A hardcoded decision tree cannot adapt to nuance, such as satire versus genuine hate, no matter how fine the labels are. The fix is model-driven reasoning.

WHERE THIS COMES FROM

Lesson 1.1: Agentic Loops (Model-driven decision-making)

Lesson 1.1: Agentic Loops (Anti-patterns)

Q04 · D2 2.4 MCP-SERVER-INTEGRATION / CONFIG-SCOPE q-2-4-010 · Developer Productivity Tools CORRECT

A team's shared `~/.mcp.json` configures a PostgreSQL MCP server. A developer adds a personal staging database MCP server to their `~/.claude.json`, so both database tools are available in their session. When

they query 'check user count', the agent calls the production tool instead of staging. What is the best resolution?

- B

Rename the staging tool to a distinct name, put its environment in the description, and add a session rule that staging tools take precedence.

Description tuning alone cannot route a context-free prompt like 'check user count' because the user gave the agent no environment cue. The fix is to make the two tools structurally distinguishable (distinct names, environment in the description) and to give the agent an explicit routing rule for the session. Renaming sits inside the developer's own ~/.claude.json so it does not affect the shared team configuration.

- C

Move the personal staging server into the shared .mcp.json beside production and let the agent pick between them from each server's connection string.

Personal staging infrastructure does not belong in the shared .mcp.json, and agents route by tool name and description, not by connection strings, so this neither isolates the config nor fixes selection.

- D

Use tool_choice with forced selection of the staging database tool for all turns during testing.

Forced selection prevents the agent from calling any other tool, including production database queries that may be needed for comparison or non-test tasks during the session.

- A

Remove the production database from .mcp.json during testing so only the staging server is available.

Modifying the shared .mcp.json affects all team members and removes production access for legitimate use. The configuration should not be altered for one developer's testing needs.

WHERE THIS COMES FROM

Lesson 2.4: MCP Server Integration (User vs project scope)

Lesson 2.1: Tool Interface Design (Differentiating similar tools)

Q05 · D2 2.5 BUILT-IN-TOOLS / GREP-VS-GLOB q-2-5-001 · Developer Productivity Tools INCORRECT

A developer needs to find all files that call a deprecated function processLegacyOrder() and also find all test files for those callers. Which tool sequence is correct?

- A

Glob for '*processLegacyOrder*' to find callers, then Grep for test files.

Glob matches file paths, not file contents. It cannot find function callers — it would only match files named after the function, which is unlikely. (your answer)

- D

Bash with 'find . -name "*.ts" | xargs grep processLegacyOrder' for both steps.

While technically functional, this bypasses the built-in tools designed for these tasks. The exam expects candidates to use the right built-in tool for each task.

- C

Grep for 'processLegacyOrder' to find the callers, then Glob to match their test files by name.

Grep searches file contents, which is correct for finding function callers. Glob matches file paths, which is correct for finding test files by naming pattern such as `**/*.test.tsx`. This is the optimal two-step sequence. (correct answer)

- B** Read every source file to search for the function calls, then Read every test file to confirm the callers.

Reading all files upfront is a context-budget killer. It consumes tokens on irrelevant files and is the exact anti-pattern the exam penalises.

WHERE THIS COMES FROM

Lesson 2.5: Built-in Tools (Grep vs Glob)

Claude Code: Built-in Tools ↗

Q06 · D1 · 1.7 SESSION-STATE-RESUMPTION / FRESH-CONTEXT · q-1-7-008 · Content Moderation and Classification System CORRECT

The moderation system handles appeals where users contest a moderation decision. Currently, the same agent that made the original decision re-evaluates the appeal. Appeal overturn rates are suspiciously low. What is the most effective architectural change?

- A** Add stronger instructions to the appeal handler's prompt requiring it to weigh the user's perspective fairly and set aside its earlier decision.

Prompt instructions cannot overcome the bias of an agent reviewing its own decision. The original reasoning context still influences the re-evaluation regardless of instructions.

- B** Route appeals to a human reviewer who has full authority to overturn automated decisions

Human review for every appeal does not scale and removes the benefit of automated moderation. Some appeals can be resolved automatically by a separate, unbiased instance.

- C** Route appeals to a separate agent instance that cannot see the original reasoning, given only the content and the user's justification.

A fresh agent instance without access to the original reasoning context evaluates the content independently. This avoids confirmation bias from the original decision. The appeal agent sees only the content and the user's argument, enabling a genuinely independent review.

- D** Automatically overturn decisions where the user provides any appeal justification to improve user trust

Automatic overturn on any appeal completely undermines moderation. Users who genuinely violated policies would simply appeal every decision.

WHERE THIS COMES FROM

Lesson 1.7: Session State and Resumption (Stale context)

Lesson 1.2: Multi-Agent Orchestration (Isolation principle)

Q07 · D1 · 1.3 SUBAGENT-INVOCATION-CONTEXT / PARALLEL-SPAWNING · q-1-3-010 ·

Content Moderation and Classification System

CORRECT

A user reports a post that contains both potentially defamatory text and an embedded image that may violate graphic content policies. The coordinator receives this report as a single moderation request. What is the correct delegation strategy?

D

Create a new combined text-and-image subagent specifically for multi-modal reports

Creating a new combined subagent duplicates capability that already exists in the specialist subagents. The coordinator's role is to decompose and delegate, not to spawn new agents for every combination of concerns.

B

Send the entire report to whichever subagent handles the more severe category

Severity cannot be determined until both analyses are complete. Routing to a single subagent also means one concern goes unanalysed.

C

Route the text to the text classifier and the image to the image analyser in parallel, then aggregate both results for the final decision.

The coordinator should decompose multi-concern requests into independent subtasks. Text defamation and image policy violations are separate analysis dimensions that can be evaluated in parallel. The coordinator aggregates the results to make a unified moderation decision.

A

Send the entire report to the text classifier first, then forward its output to the image analyser so the image step runs only after the text step has finished.

Sequential processing creates unnecessary latency when the two concerns are independent. The text and image analyses do not depend on each other and can run in parallel.

WHERE THIS COMES FROM

Lesson 1.3: Subagent Invocation and Context Passing (Parallel spawning)

Lesson 1.4: Workflow Enforcement and Handoff (Multi-concern request handling)

Q08 · D3 · 3.3 PATH-SPECIFIC-RULES / GLOB-PATTERNS · q-3-3-001 ·

CI/CD Pipeline Integration

INCORRECT

A codebase has test files co-located with source files throughout 50+ directories (e.g., Button.test.tsx next to Button.tsx). The team wants all tests to follow the same conventions regardless of location. What is the most maintainable approach?

D

Create a skill in .claude/skills/ with a paths frontmatter matching the test files so it auto-activates whenever a test file is edited

Skills can auto-activate via a paths frontmatter, but they load on demand as a task-style workflow, not as always-in-context guidance. Test conventions must shape every edit to a matching file, which is what .claude/rules/ provides: rules load into context automatically when Claude reads a matching file, with no invocation step. (your answer)

B

Place a CLAUDE.md file in every directory containing test files

With 50+ directories, this creates massive duplication and maintenance burden. Every new directory with tests would need a copy.

C Add all test conventions to the root CLAUDE.md file

Root CLAUDE.md loads for every session. Test conventions would consume tokens even when editing non-test files like API handlers or database models.

A Create a .claude/rules/ file with frontmatter paths: ["**/*.test.tsx", "**/*.test.ts"] holding the test conventions

Glob patterns in .claude/rules/ match files by pattern across the entire codebase. The conventions load automatically when editing any test file, regardless of directory. No maintenance burden as new test files are added. (correct answer)

WHERE THIS COMES FROM

Lesson 3.3: Path-Specific Rules (Path-specific rules)

Lesson 3.3: Path-Specific Rules (Practical examples)

Q09 · D1 1.2 ORCHESTRATION-PATTERNS / ISOLATION q-1-2-009 · Content Moderation and Classification System CORRECT

The content moderation system uses a hub-and-spoke architecture with a coordinator that routes posts to specialist subagents: a text classifier, an image analyser, and a policy enforcer. The team notices that the image analyser sometimes directly calls the policy enforcer's action tools to remove posts, bypassing the coordinator. What is the architectural violation and how should it be fixed?

B The image analyser breaks hub-and-spoke isolation; scope its tools to image analysis and route all results back through the coordinator.

In hub-and-spoke orchestration, subagents must be isolated and communicate only through the coordinator. The image analyser should return its classification result to the coordinator, which then decides whether to invoke the policy enforcer. Direct subagent-to-subagent communication breaks the architecture.

A Give the image analyser its own copy of the policy enforcer's action tools so it can remove posts itself without calling another subagent.

Duplicating action tools across subagents creates inconsistent enforcement and makes policy changes harder to propagate. The issue is the communication pattern, not tool availability.

D Add a message queue between the image analyser and the policy enforcer so communication is asynchronous

An async queue between subagents still bypasses the coordinator. In hub-and-spoke, all inter-agent communication must flow through the coordinator regardless of synchronicity.

C Merge the image analyser and policy enforcer into a single subagent to simplify the communication flow

Merging subagents with distinct responsibilities (analysis vs enforcement) violates separation of concerns. The image analyser and policy enforcer have fundamentally different roles and tool requirements.

WHERE THIS COMES FROM

Lesson 1.2: Multi-Agent Orchestration (Isolation principle)

Lesson 1.2: Multi-Agent Orchestration (Hub-and-spoke architecture)

Q10 · D5 · 5.5 HUMAN-REVIEW-CALIBRATION / PER-FIELD-CALIBRATION · q-5-5-005 · Structured Data Extraction Pipeline CORRECT

A contract extraction system reports 96% overall accuracy. The team plans to auto-approve all extractions where model confidence exceeds 90%. A pilot reveals party name extraction achieves 99% accuracy but indemnification clause extraction only 71%, even though the model reports high confidence on both. What should they implement before automating?

B Calibrate confidence thresholds per field type using labelled validation sets, and implement stratified sampling to continuously monitor accuracy by document type and field segment.

Field-level confidence calibration using ground truth data exposes the discrepancy between reported confidence and actual accuracy. Stratified sampling provides ongoing monitoring to detect novel error patterns. Together, these ensure automation is only applied where validated accuracy justifies it.

C Exclude indemnification clauses from automation entirely and continue automating all other field types at the 90% threshold.

This addresses the known problem but does not detect future calibration issues with other field types. Without systematic calibration and monitoring, other fields with hidden accuracy problems will be automated incorrectly.

D Train a separate classification model to predict extraction accuracy before deciding whether to automate each extraction.

A separate model adds infrastructure complexity when the solution is to calibrate the existing confidence scores against labelled data. Calibration directly addresses the unreliable confidence outputs without requiring additional ML infrastructure.

A Raise the automation confidence threshold from 90% to 99% to ensure only the most reliable extractions are automated.

Raising the threshold does not fix the calibration problem. The model reports high confidence even for indemnification clauses where it achieves only 71% accuracy. Raw confidence scores are unreliable without calibration.

WHERE THIS COMES FROM

Lesson 5.5: Human Review and Confidence Calibration (Field-level calibration)

Lesson 5.5: Human Review and Confidence Calibration (Stratified sampling)

Q11 · D2 · 2.3 TOOL-DISTRIBUTION-CHOICE / SCOPED-CROSS-ROLE · q-2-3-001 · Developer Productivity Tools CORRECT

A synthesis agent frequently returns control to the coordinator for simple fact verification, adding 2-3 round trips per task and 40% latency. Analysis shows 85% of verifications are simple lookups. What is the most effective solution?

B Give the synthesis agent a scoped verify_fact tool for simple lookups, escalating only complex checks to the coordinator.

A scoped cross-role tool handles the 85% simple case directly, eliminating round-trip latency. Complex cases still route through the coordinator for proper handling.

C Add a coordinator-level cache of verification results so repeated lookups return instantly, removing most of the round-trip latency.

Caching helps with repeated lookups but does not address the fundamental round-trip overhead for first-time verifications, which are the majority.

D Remove fact verification from the synthesis workflow to eliminate the latency entirely.

Removing verification compromises output quality. The goal is to make verification faster, not to skip it.

A Increase the coordinator's parallelism so it can process verification requests faster.

Faster processing does not eliminate unnecessary round trips. The latency comes from the routing overhead itself, not the coordinator's speed.

WHERE THIS COMES FROM

Lesson 2.3: Tool Distribution and Tool Choice (Scoped cross-role tools)

Q12 · D5 5.3 ERROR-PROPAGATION / STRUCTURED-ERROR-CONTEXT q-5-3-001

CORRECT

A web search subagent in a multi-agent research system times out while researching a complex topic. You need to design how this failure information flows back to the coordinator. Which approach best enables intelligent recovery?

A Return structured error context including failure type, attempted query, partial results, and potential alternative approaches

This gives the coordinator everything it needs to decide: retry with modified query, try an alternative approach, or proceed with partial results.

B Implement automatic retry with exponential backoff, returning a generic 'search unavailable' status only after all retries are exhausted

The generic status hides valuable context from the coordinator, preventing informed recovery decisions.

C Catch the timeout and return an empty result set marked as successful

Silent suppression prevents any recovery. The coordinator believes the search succeeded and found nothing, so it will not attempt alternatives.

D Propagate the timeout exception to a top-level handler that terminates the entire research workflow

Workflow termination wastes partial results from other subagents that may have completed successfully.

WHERE THIS COMES FROM

Lesson 5.3: Error Propagation in Multi-Agent Systems (Structured error context)

Q13 · D5 · 5.5 HUMAN-REVIEW-CALIBRATION / STRATIFIED-SAMPLING · q-5-5-009 · Structured Data Extraction Pipeline **CORRECT**

Your extraction pipeline reports 97% aggregate accuracy, and you want to automate high-confidence extractions. Which safeguards should be in place before you reduce human review? (Select 3)

E Expand the review team until every extraction can be checked by hand.

Reviewing everything ignores the point of calibration: routing limited reviewer capacity to low-confidence and ambiguous cases.

C Verify accuracy separately by document type and field segment, not just in aggregate.

A strong aggregate figure can mask poor performance on specific document types or fields.

D Keep stratified random samples of high-confidence extractions under review to measure error rates and catch novel patterns.

Stratified sampling is the guide's mechanism for ongoing measurement after automation.

A Calibrate field-level confidence thresholds against a labelled validation set.

Confidence scores only route review attention correctly once they are calibrated on labelled data.

B Rely on the aggregate accuracy figure once it has held steady for a full quarter.

Stability of the aggregate number does not reveal per-segment weaknesses, which is the risk the guide highlights.

WHERE THIS COMES FROM

Lesson 5.5: Human Review and Confidence Calibration (Aggregate metrics trap)

Lesson 5.5: Human Review and Confidence Calibration (Stratified sampling)

Lesson 5.5: Human Review and Confidence Calibration (Field-level calibration)

Q14 · D4 · 4.1 SYSTEM-PROMPTS / FALSE-POSITIVES · q-4-1-001 · CI/CD Pipeline Integration **INCORRECT**

Your CI/CD code review pipeline has a 40% false positive rate on 'documentation mismatch' findings, causing developers to ignore ALL review categories. What is the most effective fix?

A Add 'only report high-confidence documentation issues' to the system prompt

Vague confidence instructions do not improve precision. The model has no concrete criteria for what 'high-confidence' means.

B Temporarily disable the documentation mismatch category while refining its prompts with explicit criteria and code examples

This restores trust in other categories immediately while you iterate on the problematic category with specific criteria. **(correct answer)**

C Increase the model temperature to get more varied results and filter outliers

Temperature affects randomness, not precision. This would likely increase false positives.

D Add a second model pass to verify each finding before reporting

A second pass without better criteria will have the same false positive problem. Fix the criteria first. **(your answer)**

WHERE THIS COMES FROM

Lesson 4.1: System Prompts with Explicit Criteria (False-positive trust problem)

Q15 · D4 · 4.4 VALIDATION-RETRY / RETRY-BOUNDARY · q-4-4-008 · Structured Data Extraction Pipeline **INCORRECT**

A validation-retry loop still fails on the same documents after three attempts each. In which situations will further retries not help? (Select 2)

D The information exists only in an external document that is not provided in context.

Retries cannot reach data outside the model's context; the fix is supplying the external document, not another attempt. **(correct answer)**

C The validation service timed out on the last attempt.

A timeout is a transient failure, which is precisely the class of error a retry is likely to resolve.

B The required information is simply absent from the source document.

No amount of retrying makes missing information appear; the guide marks absent data as the boundary of retry effectiveness.

A The model placed a correct value in the wrong field on the last attempt.

A value in the wrong field is a semantic validation error; retry-with-error-feedback naming the misplaced field guides the next attempt to correct it.

WHERE THIS COMES FROM

Lesson 4.4: Validation, Retry, and Feedback Loops (Retry boundary)

Lesson 4.4: Validation, Retry, and Feedback Loops (Retry with error feedback)

Q16 · D5 · 5.5 HUMAN-REVIEW-CALIBRATION / STRATIFIED-SAMPLING · q-5-5-002 · Structured Data Extraction Pipeline

INCORRECT

A legal document extraction pipeline has been running for three months with calibrated confidence thresholds. Extractions above 90% confidence are auto-approved without human review, saving 60% of reviewer time. A new client submits contracts written in an unusual two-column legal format the system has not encountered before. The system reports 92% confidence on these extractions. What is the appropriate safeguard?

B Trust the calibrated confidence threshold since it was validated against labelled data and the system reports 92% confidence, which exceeds the 90% auto-approval threshold

Confidence calibration is only valid for document types represented in the calibration data. A novel format the system has never encountered can produce miscalibrated confidence scores — the model may be systematically overconfident because it does not recognise its own unfamiliarity with the format.

A Maintain stratified random sampling of high-confidence extractions across document types to catch the new format's error rate.

Stratified random sampling is designed precisely for this scenario: detecting novel error patterns in high-confidence extractions that slip past threshold-based routing. By sampling across document types, the new format's errors would surface during routine verification even though the confidence scores appear acceptable. (correct answer)

D Add a document format classifier that flags unknown formats for mandatory human review before any auto-approval

While not wrong in principle, this adds ML infrastructure complexity that stratified sampling already handles. A format classifier requires training data and maintenance, whereas ongoing stratified sampling detects novel error patterns without additional models. (your answer)

C Raise the confidence threshold from 90% to 98% for all document types to account for novel formats

Raising the threshold globally penalises well-understood document types that are genuinely accurate at 90%+. This wastes reviewer capacity on documents that do not need review, while still not guaranteeing detection of format-specific issues.

WHERE THIS COMES FROM

Lesson 5.5: Human Review and Confidence Calibration (Stratified sampling)

Q17 · D4 · 4.1 SYSTEM-PROMPTS / SEVERITY-EXAMPLES · q-4-1-002 · CI/CD Pipeline Integration

INCORRECT

Your Claude Code review prompt for the pipeline's polyglot codebase classifies severity using prose descriptions like 'critical means the code is dangerous' and 'minor means the code is slightly suboptimal'. Developers complain that identical code patterns receive different severity ratings across runs. What is the most effective improvement?

D Add a second model pass that re-evaluates each finding's severity to catch inconsistencies

A second pass using the same vague prose criteria will produce the same inconsistency. The criteria themselves must be improved with concrete code examples before adding verification layers. (your answer)

A Replace prose severity descriptions with concrete TypeScript code examples for each severity level.

Severity calibration with concrete code examples produces consistent classification across invocations. Prose descriptions are inherently ambiguous and leave the model to interpret severity differently each time. (correct answer)

B Add a confidence threshold so only findings above 90% confidence are reported, filtering out uncertain severity ratings

Self-reported confidence scores are poorly calibrated. Confidence-based filtering does not fix the root cause, which is ambiguous severity criteria. Explicit code examples are the correct fix.

C Lower the model temperature to 0 to ensure deterministic severity ratings

While lower temperature reduces randomness, it does not address the fundamental problem: the model has no concrete reference for what each severity level looks like. Code examples provide that reference.

WHERE THIS COMES FROM

Lesson 4.1: System Prompts with Explicit Criteria (Severity calibration)

Q18 · D5 5.6 INFORMATION-PROVENANCE / CONFLICT-HANDLING q-5-6-005

CORRECT

A synthesis agent receives findings from three subagents: solar investment grew 15% (Bloomberg, January 2024), solar investment grew 22% (IEA, March 2024), and cost-per-watt data. The synthesis agent currently picks the IEA figure because it is more recent. What is the correct approach?

D Flag the conflicting figures and pause report generation until a human researcher resolves the discrepancy.

Pausing for human resolution is unnecessary when the difference can be explained by temporal or methodological factors. Proper annotation with provenance resolves the ambiguity without blocking the workflow.

A Always use the most authoritative source (IEA as an international agency) and discard the Bloomberg figure.

Discarding a credible source based on perceived authority silently removes valid data. Both sources are credible, and the difference may reflect different methodologies, scopes, or time periods.

C Present both figures with source attribution and dates, noting what may explain the difference.

Preserving both data points with provenance — source, date, and potential explanations for the difference — lets the consumer interpret the data correctly. Different publication dates and methodologies explain the discrepancy without requiring the synthesis agent to judge which is 'right'.

B Average the two values (18.5%) and cite both sources to present a balanced view.

Averaging conflicting statistics is mathematically misleading. The two figures may measure different scopes (e.g., different regions or investment types), making an average meaningless. The difference needs to be preserved, not merged.

WHERE THIS COMES FROM

Lesson 5.6: Information Provenance and Multi-Source Synthesis (Conflict handling)

Lesson 5.6: Information Provenance and Multi-Source Synthesis (Temporal awareness)

Q19 · D3 · 3.5 ITERATIVE-REFINEMENT / INTERVIEW-PATTERN · q-3-5-004 · Developer Productivity Tools · CORRECT

A developer is new to a healthcare compliance domain and needs to build an audit logging system. They understand the technical implementation options but are unsure about regulatory requirements that might affect the design. Which Claude Code workflow technique is most appropriate?

D Start with direct execution using a simple implementation and iterate based on test failures

Iterating from test failures works when the requirements are known and tests can be written upfront. Healthcare compliance requirements that the developer does not know about cannot be captured in tests. Missing a regulatory requirement is not something that shows up as a test failure.

A Provide concrete input/output examples of the desired audit log format and let Claude implement it

Concrete examples fix inconsistent interpretation of known transformations. The developer's problem is not inconsistency but rather gaps in domain knowledge about regulatory requirements. Examples cannot surface unknown compliance considerations.

C Use the interview pattern so Claude asks clarifying questions about compliance requirements, data retention policies, and access control needs before proposing a design

The interview pattern is specifically designed for unfamiliar domains where the developer might miss important considerations. By having Claude ask probing questions about regulatory requirements, the developer surfaces constraints they might not have considered, leading to a more compliant design.

B Use plan mode to have Claude explore the codebase and propose multiple audit logging architectures

Plan mode is for evaluating implementation strategies when the requirements are understood. The developer's gap is in domain requirements (regulatory compliance), not in technical approach selection.

WHERE THIS COMES FROM

Lesson 3.5: Iterative Refinement Techniques (Technique hierarchy)

Lesson 3.5: Iterative Refinement Techniques (When to use each technique)

Q20 · D4 · 4.3 STRUCTURED-OUTPUT / TOOL-CHOICE-FORCING · q-4-3-002 · Structured Data Extraction Pipeline · INCORRECT

Your legal document extraction pipeline uses tool_use with tool_choice set to 'auto'. The pipeline processes contracts and sometimes receives plain text analysis instead of the structured JSON extraction you need. The team suggests switching to prompt-based JSON with explicit formatting instructions. What is the correct approach?

C Keep tool_use and set tool_choice to force the specific extraction tool by name, guaranteeing structured output on every request.

Forcing a specific tool with tool_choice {type: 'tool', name: 'extract_contract'} guarantees the model calls that exact tool every time, eliminating both text-only responses and wrong-tool selection. This is the most reliable approach for guaranteed structured output. (correct answer)

A Switch to prompt-based JSON as suggested, since the model clearly prefers text responses for these documents

Prompt-based JSON is less reliable than tool_use and can produce malformed output. The issue is tool_choice configuration, not the tool_use mechanism itself.

B Keep tool_use but switch tool_choice from 'auto' to 'any' to guarantee the model always returns a structured tool call

While 'any' guarantees a tool call, it lets the model choose which tool. If you have multiple tools defined, the model might call the wrong one. For a single extraction tool, this works, but the most precise solution is to force the specific extraction tool. (your answer)

D Add stronger instructions in the system prompt telling the model to always use the extraction tool and never respond with plain text

Instructions alone cannot guarantee tool use when tool_choice is 'auto'. The API-level tool_choice parameter is the correct mechanism for enforcing structured output, not prompt instructions.

WHERE THIS COMES FROM

Lesson 4.2: Structured Output with Tool Use (tool_choice forcing)

Anthropic: Tool Use Documentation >

Q21 · D5 5.2 ESCALATION-AMBIGUITY / EXPLICIT-ESCALATION q-5-2-005

INCORRECT

A customer says: 'This is ridiculous, I have been waiting 20 minutes. Just connect me to a real person.' The agent has access to the customer's account and can see the issue is a simple password reset that takes 30 seconds to resolve. What should the agent do?

D Ask the customer if they would prefer to wait for a human or let the agent resolve the password reset quickly.

The customer has already stated their preference clearly. Asking again ignores their explicit request and adds friction to an already frustrating experience. (your answer)

A Acknowledge the frustration and offer to resolve the password reset, explaining it will take only 30 seconds — the issue is straightforward and escalation is unnecessary.

The customer has explicitly requested a human agent. Regardless of how simple the issue is, the agent must honour the explicit request immediately. Attempting to resolve first ignores the customer's stated preference.

C Run sentiment analysis to confirm the customer is genuinely frustrated before deciding whether to escalate or attempt resolution.

Sentiment analysis is an unreliable escalation trigger. More importantly, the customer's explicit request for a human is the trigger here, not their emotional state. Sentiment analysis adds unnecessary processing.

B Escalate to a human agent immediately without attempting to resolve the issue, because the customer explicitly requested a human.

When a customer explicitly says 'connect me to a real person', the agent must escalate immediately. This is one of the three valid escalation triggers. The simplicity of the issue is irrelevant — the customer's explicit request takes priority.
(correct answer)

WHERE THIS COMES FROM

Lesson 5.2: Escalation and Ambiguity Resolution (Valid escalation triggers)

Lesson 5.2: Escalation and Ambiguity Resolution (Frustration nuance)

Q22 · D4 · 4.4 VALIDATION-RETRY / RETRY-WITH-ERROR q-4-4-002 · Structured Data Extraction Pipeline **CORRECT**

Your extraction pipeline retries failed documents, but the retry simply resends the original document with the same prompt. Success rates on retries are only marginally better than the first attempt. Developers have identified that most failures are format mismatches where the model places values in the wrong fields. How should you improve the retry mechanism?

A Increase the number of retries from 1 to 3, since extraction is non-deterministic and more attempts improve odds

Naive retries without error feedback produce the same mistakes. More attempts at the same flawed approach yield diminishing returns. The model needs to see what went wrong.

C Skip retries and route all failures directly to human review to avoid wasting API costs

Format mismatches are fixable errors. The model can self-correct when given proper error feedback. Skipping retries wastes the model's self-correction capability for errors that are resolvable.

D Send the original document, the failed extraction, and the validation error naming the misplaced fields.

Retry-with-error-feedback is dramatically more effective than naive retries. Sending the original document, the failed extraction, and the specific validation error allows the model to self-correct. Format mismatches are fixable errors that respond well to this approach.

B Switch to a different model for retries so a fresh perspective catches the errors

The issue is not the model but the absence of error context. Any model retrying without seeing the specific validation error will likely repeat the same mistakes.

WHERE THIS COMES FROM

Lesson 4.4: Validation, Retry, and Feedback Loops (Retry with error feedback)

Lesson 4.4: Validation, Retry, and Feedback Loops (Self-correction flow)

Q23 · D1 · 1.4 WORKFLOW-ENFORCEMENT-HANDOFF / CONFIDENCE-ROUTING · q-1-4-007 ·

Content Moderation and Classification System

CORRECT

The moderation system auto-removes posts classified as policy violations. Production logs reveal that 3% of auto-removed posts were legitimate content (false positives), generating user complaints and eroding platform trust. The team considers two approaches: (A) escalate all borderline cases to human reviewers, or (B) add a confidence threshold where only high-confidence violations are auto-removed and everything else goes to human review. Which approach is better and why?

C Neither — instead lower the classification threshold so fewer posts are flagged as violations

Lowering the threshold reduces false positives but increases false negatives (genuine violations going undetected). This trades one problem for another rather than adding an appropriate escalation path.

A Approach A is better because human reviewers are always more accurate than automated systems

Escalating all borderline cases overwhelms the human review queue without leveraging the system's ability to handle clear-cut cases autonomously. The goal is to route intelligently, not to defer everything.

B Approach B: auto-remove only high-confidence violations, escalate uncertain cases to human review, and auto-approve high-confidence safe content.

Confidence-based routing creates a tiered system: high-confidence violations are actioned immediately, high-confidence safe content passes through, and uncertain cases receive human review. This balances speed, accuracy, and reviewer workload while directly addressing the false positive problem.

D Approach B, but apply a single fixed 95% confidence threshold to every category regardless of how costly a false positive is for that content type.

A single fixed threshold across all categories ignores that violation types have different risk profiles. The threshold should be calibrated per category based on the cost of false positives versus false negatives.

WHERE THIS COMES FROM

Lesson 1.4: Workflow Enforcement and Handoff (Confidence-based escalation)

Lesson 1.4: Workflow Enforcement and Handoff (Enforcement spectrum)

Q24 · D4 · 4.2 FEW-SHOT-PROMPTING / FEW-SHOT-STYLE · q-4-2-002 · CI/CD Pipeline Integration

CORRECT

Your Claude Code agent generates unit tests in the CI pipeline. When given detailed instructions alone, it produces tests with inconsistent assertion styles: sometimes using `expect().toBe()`, sometimes `assert.equal()`, and occasionally mixing both in the same file. Adding more detailed instructions about assertion style did not fix the problem. What should you do next?

B Add a linter post-processing step to automatically convert all assertions to a single style

Post-processing masks the problem rather than solving it. The model should learn to produce consistent output directly, which few-shot examples achieve more effectively.

D Add 2-4 few-shot examples demonstrating the desired assertion style with reasoning for each testing decision, covering edge cases like async functions and error handling

Few-shot examples are the most effective technique when detailed instructions alone produce inconsistent formatting. Including 2-4 examples with reasoning teaches generalisation to novel patterns, not just pattern-matching. Covering varied scenarios (async, error handling) ensures the model generalises correctly.

C Add 2-4 few-shot examples showing complete test files with the desired assertion style and reasoning for why that style was chosen over alternatives

This is partially correct but incomplete. The examples only cover the assertion style preference. Without covering varied scenarios (async functions, error handling), the model pattern-matches the specific cases shown rather than generalising the style consistently across all test types.

A Switch to a different model that better follows formatting instructions

The issue is not model capability. When detailed instructions alone produce inconsistent formatting, the correct intervention is few-shot examples, not a model change.

WHERE THIS COMES FROM

Lesson 4.2: Few-Shot Prompting (Few-shot examples)

Anthropic: Multishot (Few-Shot) Prompting ↗

Q25 · D4 · 4.4 VALIDATION-RETRY / PYDANTIC-VALIDATION · q-4-4-010 · Structured Data Extraction Pipeline **INCORRECT**

The pipeline validates each `tool_use` extraction against a Pydantic model with `model_validate()`. Validation succeeds and every field has the correct type, yet extracted totals still disagree with the documents' stated totals. What explains this, and what is the fix?

C The schema needs numeric minimum and maximum constraints on the `stated_total` field so that mismatched totals are rejected at parse time

A per-field range cannot express a relationship between two fields; the mismatch is only detectable by comparing them in validation code.

B Type checks passed but no rule relates the totals; add a model validator for the sum rule and route its errors into retries

Schema conformance and semantic correctness are different layers; the sum relationship must be written as a validator, whose errors drive the retry loop. (correct answer)

D Semantic disagreement indicates hallucination; lower the temperature so extraction becomes deterministic

Sampling settings do not create correctness, and determinism would only repeat the same wrong extraction; this failure class belongs to semantic validation. (your answer)

A model_validate() is failing silently; pin an older Pydantic version until the regression is fixed

model_validate() did its job: the extraction conforms to the model's types. Nothing failed silently; the missing check was never defined.

WHERE THIS COMES FROM

Lesson 4.4: Validation, Retry, and Feedback Loops (Pydantic as the validation layer)

Q26 · D4 · 4.5 BATCH-PROCESSING / CUSTOM-ID · q-4-5-004 · CI/CD Pipeline Integration CORRECT

Your CI/CD pipeline processes code reviews using the Message Batches API. After a batch completes, you need to match each review result back to the corresponding pull request. What is the correct mechanism for correlating batch results with their original requests?

B Rely on the batch API returning results in the same order they were submitted

Batch results are not guaranteed to return in submission order. The custom_id field is the correct correlation mechanism.

C Use the custom_id field in each batch request to store the pull request identifier, then match results by custom_id

The custom_id field exists specifically for correlating batch request/response pairs. Setting it to the pull request identifier provides reliable, direct correlation without relying on result ordering or content parsing.

D Submit each pull request as a separate batch of one item to maintain correlation through the batch ID

Submitting individual batches defeats the purpose of batch processing and does not leverage the custom_id mechanism designed for exactly this use case.

A Parse the review content to identify which pull request it refers to based on file names mentioned in the review

Content parsing is fragile and error-prone. The Batches API provides a built-in mechanism for result correlation.

WHERE THIS COMES FROM

Lesson 4.5: Batch Processing and Prompt Optimisation (custom_id matching)

Anthropic: Message Batches API ·

Q27 · D1 · 1.2 ORCHESTRATION-PATTERNS / COORDINATOR · q-1-2-002

CORRECT

A consulting firm's multi-agent research system has a coordinator that always invokes the full pipeline of five subagents (web search, document analysis, data extraction, synthesis, and formatting) for every query, including simple factual lookups that only need web search. This adds unnecessary latency and cost. What is the correct architectural fix?

D Split into two separate systems: one for simple queries and one for complex queries

Maintaining two separate systems creates unnecessary complexity. The coordinator's role is precisely to analyse query requirements and make routing decisions. A single coordinator with dynamic selection handles both cases.

A Have the coordinator select which subagents to invoke per query, not always the full pipeline.

The coordinator should analyse query requirements and decide which subagents are needed. A simple factual lookup only needs the web search agent, not the full five-agent pipeline. Dynamic selection reduces latency and cost for straightforward requests.

B Allow subagents to communicate directly with each other so they can skip unnecessary steps in the pipeline

Direct subagent communication violates the hub-and-spoke architecture. All communication must flow through the coordinator. The fix is smarter coordinator routing, not bypassing the coordinator.

C Add a caching layer so subagents that have no work to do return immediately

Caching does not address the architectural issue. Invoking subagents that are not needed adds API call overhead and latency regardless of whether results are cached. The coordinator should not invoke unnecessary subagents in the first place.

WHERE THIS COMES FROM

Lesson 1.2: Multi-Agent Orchestration (Coordinator responsibilities)

Q28 · D5 · 5.5 HUMAN-REVIEW-CALIBRATION / REVIEWER-CAPACITY · q-5-5-007

CORRECT

Claude Code generates API reference pages for 150 endpoints across three categories: payment processing (30 endpoints, high regulatory risk), internal tooling (80 endpoints, low risk), and public data queries (40 endpoints, moderate risk). The team cannot manually review all 150. How should they structure human review?

A Randomly sample 15% of all endpoints (approximately 23 pages) for review, ensuring a representative cross-section

Uniform random sampling treats all endpoints equally, but payment processing endpoints carry much higher risk. A 15% sample might only include 4-5 payment endpoints, inadequately covering the highest-risk category.

C Review only the payment processing endpoints since they are the highest risk, and trust Claude Code's output for the remaining 120 endpoints

Completely skipping review for 120 endpoints risks undetected errors in public-facing documentation. Even low-risk internal tooling docs should have some sample review to catch systematic generation issues that might affect all categories.

B Use stratified sampling: review 100% of payment docs, 10% of public data query docs, and 5% of internal tooling docs.

Stratified sampling allocates review effort proportional to risk. Payment processing documentation has regulatory implications warranting full review. Public data queries affect external users and deserve moderate scrutiny. Internal tooling has the lowest blast radius and can tolerate the lightest review. This achieves thorough coverage of critical content within a manageable review budget.

D Have Claude Code self-assess each generated page with a confidence score and only review pages where confidence falls below 80%

LLM self-reported confidence is poorly calibrated. Claude Code may report high confidence on pages with subtle factual errors (e.g., incorrect parameter types, wrong default values) because the generated text is fluent and internally consistent.

WHERE THIS COMES FROM

Lesson 5.5: Human Review and Confidence Calibration (Reviewer capacity)

Lesson 5.5: Human Review and Confidence Calibration (Stratified sampling)

Q29 · D3 3.6 CI/CD-INTEGRATION / SESSION-ISOLATION q-3-6-005 · CI/CD Pipeline Integration CORRECT

A CI/CD pipeline runs three Claude Code steps in sequence: (1) generate a changelog from git commits, (2) review the changelog for accuracy, and (3) check for breaking changes. The team notices that step 2 never flags inaccuracies and step 3 misses obvious breaking changes that the changelog omits. What is the root cause?

C The -p flag is not being used, causing each step to wait for interactive input

If -p were missing, the pipeline would hang rather than produce output that misses issues. The steps are producing output (the changelog is generated, reviews complete), but the reviews are ineffective.

D The steps need to use --output-format json so that each step can parse the previous step's structured output

Output format affects how results are structured, not whether reviews are thorough. JSON output would help with parsing but would not address the fundamental issue of review steps retaining generation context bias.

B The CLAUDE.md file does not contain changelog formatting standards, so the review step has no criteria to evaluate against

Missing formatting standards would affect the quality of the generated changelog, not the review's ability to catch inaccuracies. The pattern of reviews consistently missing issues points to shared-context bias, not missing criteria.

A The three steps share session context, so steps 2 and 3 inherit step 1's reasoning instead of judging the changelog

When review and analysis steps share context with the generation step, they inherit the original reasoning and are less likely to identify gaps or errors. Independent sessions force each step to evaluate the changelog from scratch without bias from the generation reasoning. Session context isolation is critical for CI/CD review pipelines.

WHERE THIS COMES FROM

Lesson 3.6: CI/CD Integration (Session context isolation)

Q30 · D5 5.5 HUMAN-REVIEW-CALIBRATION / AGGREGATE-METRICS-TRAP q-5-5-004 · Structured Data Extraction Pipeline CORRECT

A structured data extraction system processes invoices, purchase orders, and contracts. The monitoring dashboard shows 97% overall extraction accuracy and the team considers the system production-ready. A detailed audit reveals: invoices 99.5% accuracy (80% of volume), purchase orders 98% accuracy (15% of volume), contracts 72% accuracy (5% of volume). What does this reveal and what action is required?

A The system is performing well. 72% on contracts is acceptable since contracts represent only 5% of volume and barely affect the overall metric

Acceptability depends on the business impact, not volume percentage. Contract extraction errors may have severe financial or legal consequences despite low volume. A 28% error rate on any document type is significant.

B The aggregate metric is masking a severe per-type disparity. Give contracts mandatory review and report per-type metrics.

This is a textbook case of aggregate metrics hiding per-type problems. 97% overall is misleading because high-volume invoice accuracy (99.5%) overwhelms the poor contract performance (72%). Per-type metrics would have surfaced this immediately. Contracts need different treatment until the extraction quality is acceptable.

C The training data needs rebalancing so that contracts represent a larger proportion, which will naturally improve contract accuracy

LLMs used for extraction are not retrained on production data this way. The issue is monitoring visibility and operational response, not model training data distribution.

D The 97% threshold should be raised to 99% overall to force improvement across all document types

Raising the aggregate threshold does not address per-type disparities. Even at 99% overall, contract accuracy could remain low if invoice volume continues to dominate the metric. The solution is per-type metrics, not a higher aggregate bar.

WHERE THIS COMES FROM

Lesson 5.5: Human Review and Confidence Calibration (Aggregate metrics trap)

Q31 · D1 1.4 WORKFLOW-ENFORCEMENT-HANDOFF / SEQUENCING q-1-4-006 · Developer Productivity Tools CORRECT

A coordinator using the Claude Agent SDK delegates a refactoring task to a code reviewer, a test generator, and a documentation writer in parallel, all working from the original code. The code reviewer recommends significant changes; once applied, the tests and documentation generated against the old API become invalid. What architectural fix prevents this?

B Run the code reviewer first, then the test generator and doc writer in parallel on the reviewed code.

The code reviewer's output changes the code that tests and documentation must target. Therefore, the code reviewer must run first. Once the reviewed code is stable, the test generator and documentation writer can run in parallel since they are independent of each other. This is a hybrid pattern: sequential then parallel.

D Instruct the coordinator to always run the three agents strictly one at a time, in sequence, regardless of whether their tasks actually depend on each other.

Always-sequential execution is overly conservative. The test generator and documentation writer do not depend on each other, so they can run in parallel after the code reviewer completes. Parallelise independent tasks while respecting real dependencies.

A Have all three agents share memory so they can see each other's outputs in real time

Subagents do not share memory — this is a fundamental architectural constraint. Even if they could, real-time sharing would create race conditions and ordering dependencies that are hard to manage.

C Add a validation step at the end that checks whether tests and documentation match the final code

Post-hoc validation detects the problem but does not prevent it. The tests and documentation must be rewritten entirely, wasting the initial parallel execution. The correct fix is to order the dependencies properly from the start.

WHERE THIS COMES FROM

Lesson 1.4: Workflow Enforcement and Handoff (Decision rule: sequential vs parallel)

Lesson 1.2: Multi-Agent Orchestration (Coordinator responsibilities)

Q32 · D4 · 4.2 FEW-SHOT-PROMPTING / FEW-SHOT-AMBIGUITY · q-4-2-007 · Structured Data Extraction Pipeline **CORRECT**

An invoice-extraction pipeline mislabels fields on documents where the correct interpretation is ambiguous. Which few-shot practices should you apply? (Select 2)

C Replace the examples with a longer natural-language description of each field.

Descriptions alone are what produced the inconsistency; examples communicate the expected mapping more reliably than more prose.

A Add an example for every document variant seen in production so far.

Volume is not the fix; a handful of targeted examples for the ambiguity beats exhaustive coverage and its context cost.

B Use examples showing correct extraction from varied document structures to fix empty or null extraction of required fields.

Varied-structure examples fix empty or null extraction of fields that are present, which is the few-shot remedy for unreliable extraction; fabrication of absent data is handled by nullable schema fields instead.

D Create two to four targeted examples for the ambiguous cases, showing why one interpretation is chosen.

Targeted examples that demonstrate the reasoning are the guide's specific remedy for ambiguous cases.

WHERE THIS COMES FROM

Lesson 4.2: Few-Shot Prompting (Few-shot construction)

Anthropic: Multishot (Few-Shot) Prompting ↗

Q33 · D4 · 4.1 SYSTEM-PROMPTS / KEYWORD-OVERLAP · q-4-1-005 · CI/CD Pipeline Integration **INCORRECT**

A CI/CD system prompt defines two review categories with the instructions 'Check for security vulnerabilities in each function' and 'Check for performance issues in each loop'. The model frequently calls 'performance_check' for security issues found inside loops, and 'security_check' for performance issues in security-sensitive functions. What is the root cause and best fix?

A The model is confused because loops can have both security and performance issues; add a rule that security always takes priority over performance

Priority rules add complexity without fixing the root cause. The model is confusing tools because the instruction keywords overlap with both tool names and each other.

C Add more detailed tool descriptions explaining exactly when each tool should be called

More detailed descriptions help but do not fix the root cause if the instruction text still contains overlapping keywords that cue the wrong tool. **(your answer)**

D Force tool_choice to 'auto' and let the model determine the correct tool based on the content of each finding

tool_choice 'auto' is likely already the setting. The problem is that the model's tool selection is being misled by keyword overlap in the prompt, not by the tool_choice configuration.

B Keyword overlap between the instruction phrasing and the tool names causes the confusion; rewrite them to use distinct, non-overlapping terms.

When instruction text like 'check for security in each function' overlaps with tool name patterns, the model conflates instruction keywords with tool selection cues. Using distinct terminology for instructions versus tool names eliminates this cross-contamination. **(correct answer)**

WHERE THIS COMES FROM

Lesson 4.1: System Prompts with Explicit Criteria (Keyword overlap)

Anthropic: Tool Use Documentation ↗

Q34 · D4 · 4.2 FEW-SHOT-PROMPTING / FEW-SHOT-CODED-CONTENT · q-4-2-003 ·

Content Moderation and Classification System

CORRECT

The moderation system classifies hate speech accurately for explicit slurs but misses coded language and dog-whistle terms that human moderators easily recognise. The prompt includes detailed written rules about coded language patterns. What intervention would most improve detection of coded hate speech?

C

Add 20+ examples covering every known category of coded hate speech to maximise coverage

Laundry lists of examples dilute attention and teach pattern-matching rather than generalisation. 2-4 targeted examples with reasoning are more effective than 20 examples without reasoning because they teach the analytical process.

D

Increase the model's context window so it can consider more of the user's post history for context

Context window size is not the bottleneck. The model fails to recognise coded language in individual posts, which is a prompt engineering issue, not a context limitation.

A

Add a comprehensive dictionary of every known coded term and dog-whistle to the system prompt so the classifier can match posts against the full list.

A static dictionary becomes outdated immediately as coded language constantly evolves. It also bloats the prompt without teaching the model to recognise the underlying patterns of coded speech.

B

Add 2-4 few-shot examples of coded hate speech, with reasoning naming the coded language and the targeted group.

Few-shot examples with reasoning tags teach the model the pattern-recognition process, not just individual terms. Showing the reasoning chain — surface meaning, coded meaning, targeted group, contextual signals — enables the model to generalise to new coded terms it has not seen before.

WHERE THIS COMES FROM

Lesson 4.2: Few-Shot Prompting (Few-shot for nuanced detection)

Anthropic: Multishot (Few-Shot) Prompting ↗

Q35 · D3 · 3.3 PATH-SPECIFIC-RULES / GLOB-PATTERNS · q-3-3-009 · Developer Productivity Tools · CORRECT

A developer working on a React Native project notices that Claude Code applies web-specific React conventions (e.g., using div elements) when editing mobile components in the src/mobile/ directory, despite having mobile-specific conventions documented. The root CLAUDE.md contains general React standards, and there is no directory-level or rules-based configuration for mobile. What is the best fix?

C

Replace the root CLAUDE.md web React conventions with mobile React Native conventions

Replacing web conventions with mobile conventions would break guidance for the web portion of the codebase. Both web and mobile conventions are needed, but scoped to their respective file paths.

A

Add mobile-specific conventions to the root CLAUDE.md alongside the existing web React standards

Adding mobile conventions to the root CLAUDE.md would load them for every session, including when editing web components, API handlers, or any other file. This wastes tokens and could cause confusion when both web and mobile conventions are present simultaneously.

D Create a CLAUDE.md file in src/mobile/ and remove all React standards from the root CLAUDE.md

A directory-level CLAUDE.md in src/mobile/ would work for that directory, but removing React standards from the root CLAUDE.md would leave web components without guidance. The correct approach preserves root-level standards and adds path-scoped mobile rules.

B Create a .claude/rules/react-native.md file with paths: ["src/mobile/**/*"] containing the mobile-specific conventions

A rules file with a path restriction to src/mobile/ ensures mobile conventions load only when editing mobile components. This keeps the root CLAUDE.md focused on universal standards while providing targeted, token-efficient mobile guidance.

WHERE THIS COMES FROM

Lesson 3.3: Path-Specific Rules (Rule file examples)

Lesson 3.3: Path-Specific Rules (Path-specific rules)

Q36 · D3 3.5 ITERATIVE-REFINEMENT / BATCH-VS-SEQUENTIAL q-3-5-002 · Developer Productivity Tools **INCORRECT**

A developer is iterating on a data transformation function with Claude Code and has identified three issues: (1) date parsing mishandles timezone offsets, (2) currency formatting uses the wrong locale, and (3) the output JSON schema has an incorrect field name. Issues 1 and 2 are independent; issue 3 changes the output shape both must conform to. How should the developer provide feedback?

D Use the interview pattern to have Claude ask clarifying questions about all three issues before making any changes

The interview pattern is for unfamiliar domains where the developer might miss considerations. Here, the developer has already identified the three specific issues and understands their interdependencies. The question is about feedback sequencing, not domain exploration. (your answer)

A Send all three issues in a single message to let Claude address them holistically

Batching everything together is appropriate when all issues are interdependent. Here, issues 1 and 2 are independent of each other, and sending them together risks conflating the feedback for those fixes. The interdependency is only between issue 3 and the other two.

B Send each issue in a separate sequential message, waiting for confirmation after each fix

Pure sequential iteration is appropriate when all issues are independent. Here, issue 3 (schema field name) affects the expected output that issues 1 and 2 must conform to. Fixing issues 1 and 2 first without correcting the schema would produce fixes targeting the wrong field name.

C Fix issue 3 (schema field name) first, then address issues 1 and 2 one at a time since they are independent

Issue 3 changes the output schema that issues 1 and 2 must conform to, so it must be resolved first. Once the schema is correct, issues 1 and 2 are independent, so they are fixed one at a time rather than batched, since batching independent issues can confuse which feedback applies to which fix. This follows the principle: resolve dependencies first, batch when fixes interact, and sequence when independent. **(correct answer)**

WHERE THIS COMES FROM

Lesson 3.5: Iterative Refinement Techniques (Batch vs sequential)

Q37 · D3 · 3.6 CI/CD-INTEGRATION / OUTPUT-FORMAT · q-3-6-004 · CI/CD Pipeline Integration **CORRECT**

A team wants their CI pipeline to run Claude Code for both PR code review and automated test generation. The PR review should output structured JSON for integration with their review dashboard, while the test generation should produce standard text output. How should the two pipeline steps be configured?

A Run both steps with -p and configure the review dashboard to parse plain text output from both steps

Plain text output requires fragile parsing logic in the review dashboard. The --output-format json flag provides structured, machine-readable output that is far more reliable for programmatic integration.

C Run both steps in a single Claude Code session with -p, using different prompts to request different output formats

Running both steps in a single session violates session context isolation. The review step would retain reasoning context from test generation, reducing review effectiveness. Each step should be an independent invocation.

B Run the review step with -p --output-format json and the test generation step with -p only, as separate non-interactive invocations

Each step uses -p for non-interactive mode. The review step adds --output-format json for structured output that the dashboard can parse reliably. The test generation step uses default text output since it produces code files, not structured data. Running them as separate invocations ensures session context isolation.

D Run both steps with --output-format json and have the test generation step extract code from the JSON response

While this would work technically, it adds unnecessary complexity to the test generation step. Using JSON output format only for the step that needs structured parsing (review) is the cleaner approach.

WHERE THIS COMES FROM

Lesson 3.6: CI/CD Integration (Structured output)

Lesson 3.6: CI/CD Integration (-p flag)

Claude Code: Headless / CLI Reference >

Q38 · D2 · 2.5 BUILT-IN-TOOLS / GREP-THEN-READ · q-2-5-005 · Developer Productivity Tools **CORRECT**

A developer is investigating a production bug. They know the error message 'InvalidStateTransition' is logged somewhere in the codebase, but they do not know which files contain it or where the state machine is defined. What is the correct tool sequence to locate the bug?

- C

Use Read on common file locations like src/index.ts and src/app.ts to find the error manually.

Guessing file locations is unreliable and wastes context tokens. The developer does not know where the error is defined, so a content search (Grep) is needed first.
- A

Use Glob for '**/*state*' to find state machine files, then Read each one to search for the error message.

Glob matches file paths, so it would find files with 'state' in the name, but the error string could appear in files without 'state' in their name. Reading every matched file wastes context tokens on files that may not contain the error.
- B

Grep for 'InvalidStateTransition' across the codebase, then Read the matching files.

Grep searches file contents, so it locates every occurrence of the error string regardless of file name. Read then loads only the matching files to understand the surrounding code and state machine logic. This is the minimal, targeted approach.
- D

Use Edit to search for 'InvalidStateTransition' and replace it with a more descriptive error message.

Edit is a modification tool, not a search tool. Using it to locate code is incorrect, and modifying error messages without understanding the bug first could mask the real issue.

WHERE THIS COMES FROM

Lesson 2.5: Built-in Tools (Grep for content)

Lesson 2.5: Built-in Tools (Read for files)

Q39 · D4 · 4.4 VALIDATION-RETRY / RETRY-BOUNDARY · q-4-4-001 · Structured Data Extraction Pipeline CORRECT

Your extraction pipeline validates that line item amounts sum to the stated total. For Document A, the sum is £450 but the stated total is £500. For Document B, the 'department' field is missing entirely from the source text. Which retry strategy is correct?

- B

Retry Document A with the discrepancy error; flag Document B for human review since the information is absent from the source

Document A has a fixable discrepancy (the model may have missed a line item). Document B has genuinely absent information — retries are ineffective.
- D

Skip retries for both and flag them for human review

Document A's discrepancy is likely fixable with error feedback. Skipping the retry wastes the model's self-correction capability.

C Retry both documents with the same prompt since extraction is non-deterministic

Non-determinism does not create information that is absent. Document A may benefit from retry; Document B will not.

A Retry both: send the validation error for Document A and instruct the model to find the missing department for Document B

Document B's department is absent from the source. Retrying will not make information appear that does not exist.

WHERE THIS COMES FROM

Lesson 4.4: Validation, Retry, and Feedback Loops (Retry effectiveness boundary)

Q40 · D5 5.6 INFORMATION-PROVENANCE / CLAIM-SOURCE-MAPPING q-5-6-003 · Structured Data Extraction Pipeline **CORRECT**

A data extraction pipeline extracts financial figures from annual reports. The pipeline cites sources using inline text references like 'According to page 12 of the 2024 Annual Report, revenue was \$4.2B.' When the extracted data is consumed by a downstream analytics system, the source attribution is consistently lost. What is the root cause and fix?

A The downstream system strips text formatting. Fix by making the citations bold or using a special delimiter

The problem is not formatting but data structure. Regardless of text formatting, inline prose citations are lost when text is parsed, summarised, or restructured by downstream systems.

C The downstream system needs to be modified to preserve all input text verbatim without any transformation

Requiring downstream systems to preserve verbatim text is impractical and fragile. Systems legitimately need to transform, aggregate, and restructure data. Attribution should be in a format that survives transformation.

B Inline prose citations are fragile. Fix by outputting structured claim-source mappings pairing each value with source, page, and date.

Structured claim-source mappings survive downstream processing because they are data fields, not prose. An analytics system can consume {'value': '4.2B', 'source': '2024 Annual Report', 'page': 12} without losing attribution, whereas it will discard 'According to page 12...' during text processing.

D Add a separate attribution database keyed by value hash that the downstream system queries to look up the source for any extracted figure.

While a database could store attributions, this adds unnecessary infrastructure when the extraction pipeline can simply output structured mappings directly. The attribution should travel with the data, not require a separate lookup.

WHERE THIS COMES FROM

Lesson 5.6: Information Provenance and Multi-Source Synthesis (Structured claim-source mappings)

Q41 · D3 · 3.4 PLAN-MODE-EXECUTION / HYBRID · q-3-4-005 · Developer Productivity Tools CORRECT

A team is debugging a complex distributed system issue. The lead developer wants to use Claude Code to explore logs, trace request flows, and form hypotheses without making any changes to the codebase. Partway through the investigation, they identify a one-line fix in a configuration file and want to apply it immediately. What is the optimal workflow?

A Start in plan mode for investigation, then switch to direct execution for the one-line fix once identified

Plan mode is ideal for the investigation phase: exploring logs, tracing flows, and evaluating multiple hypotheses without committing to changes. Once the fix is identified and well-understood (a clear-scope, single-file change), switching to direct execution applies it efficiently without unnecessary planning overhead.

B Use direct execution throughout, with detailed upfront instructions describing both the investigation process and potential fix patterns

Direct execution is inappropriate for the investigation phase because the problem and solution are not yet understood. Providing detailed upfront instructions assumes knowledge of the issue that the investigation is meant to discover.

C Use plan mode for the entire session, including the one-line fix, to maintain consistency

While plan mode is correct for investigation, continuing to use it for a well-understood one-line fix adds unnecessary overhead. Once the fix is identified and scoped, direct execution is the appropriate mode for a clear, limited change.

D Use allowedTools to restrict to read-only tools for investigation, then start a new session with write permissions for the fix

Starting a new session for the fix would lose all the investigation context (log analysis, request tracing, hypothesis formation) that led to identifying the fix. Switching modes within the same session preserves this valuable context.

WHERE THIS COMES FROM

Lesson 3.4: Plan Mode vs Direct Execution (Hybrid plan then execute)

Q42 · D1 · 1.3 SUBAGENT-INVOCATION-CONTEXT / PARALLEL-SPAWNING · q-1-3-011 · Content Moderation and Classification System CORRECT

The coordinator receives a batch of 200 flagged posts to moderate. Currently it processes them sequentially, taking 45 minutes. Each post’s moderation is independent — the decision on one post does not affect others. How should the coordinator handle this batch?

D Increase the iteration cap on the agentic loop to allow the agent more time to process all 200 posts

The iteration cap controls how many tool calls the agent can make within a single task, not how many independent tasks it can process. The issue is parallelism, not loop duration.

B Delegate independent posts to parallel subagent instances, with the coordinator aggregating results as they complete

Since each post's moderation is independent, the coordinator should delegate them to parallel instances. This reduces total processing time from sequential (45 minutes) to roughly the time of the slowest individual moderation, and a failure on one post does not block others.

C Split into fixed batches of 20 posts and process each batch sequentially in a single prompt

Batching 20 posts into a single prompt still risks attention dilution across posts within each batch. And sequential batch processing still takes much longer than parallel individual processing.

A Process all 200 posts in a single API call by concatenating them into one large prompt

Concatenating 200 posts into a single prompt causes attention dilution. Quality degrades for posts later in the sequence, and a single failure blocks the entire batch.

WHERE THIS COMES FROM

Lesson 1.3: Subagent Invocation and Context Passing (Parallel spawning)

Q43 · D2 2.3 TOOL-DISTRIBUTION-CHOICE / TOOL-CHOICE q-2-3-006 · Developer Productivity Tools INCORRECT

A team's agent must generate a weekly report on the first turn of each conversation by calling 'generate_report'. With 'tool_choice: 'auto'', the agent sometimes responds with a text summary instead. The team wants to guarantee 'generate_report' is called on turn one while allowing normal tool selection afterwards. What is the correct configuration?

D Remove all tools except generate_report from the first turn's tool list, then add the remaining tools for subsequent turns.

While this would work, dynamically modifying the tool list between turns is unnecessarily complex. Forced tool_choice selection achieves the same guarantee with a simpler configuration change. (your answer)

B Set tool_choice to forced selection of generate_report for the first turn, then switch to 'auto' for subsequent turns.

Forced selection with the specific tool name guarantees generate_report is called on the first turn. Switching to 'auto' afterwards restores normal behaviour where the model decides whether and which tools to call. (correct answer)

A Set tool_choice to 'any' for all turns so the agent always calls a tool.

'any' forces a tool call but does not guarantee which tool is called. The agent might call a different tool on the first turn. Additionally, forcing tool calls on every subsequent turn prevents normal text responses.

C Add a system prompt instruction stating 'Always call generate_report first' and keep tool_choice on 'auto'.

System prompt instructions are advisory, not enforced. With 'auto', the model can still choose to respond with text instead of calling the tool. This does not guarantee the mandatory first step.

WHERE THIS COMES FROM

Lesson 2.3: Tool Distribution and Tool Choice (Forced tool selection)
Anthropic: Tool Use Documentation >

Q44 · D2 · 2.5 BUILT-IN-TOOLS / EDIT-REPLACE-ALL · q-2-5-004 · Developer Productivity Tools INCORRECT

A developer needs to rename a variable from 'userData' to 'customerData' in a specific file. The variable appears 12 times throughout the file. What is the most efficient approach using Claude Code's built-in tools?

A Use Grep to find all 12 occurrences, then call Edit 12 times, once for each occurrence with unique surrounding context.

Calling Edit 12 times is inefficient. The Edit tool's replace_all parameter is designed for exactly this use case — renaming a variable across an entire file in a single call.

D Use Bash with sed to perform a global find-and-replace across the file.

While sed can perform the replacement, the exam expects candidates to use built-in tools for file modifications. Edit with replace_all is the purpose-built tool for this task. (your answer)

C Use Edit with replace_all set to true, specifying 'userData' as the old string and 'customerData' as the new string.

The Edit tool's replace_all parameter replaces all occurrences of the specified string in a single operation. This is the most efficient approach for renaming a variable throughout a file. (correct answer)

B Use Read to load the file, then Write the entire file back with all occurrences manually changed.

Read + Write rewrites the entire file, which is wasteful for a simple rename. The Edit tool with replace_all handles this more efficiently by sending only the diff.

WHERE THIS COMES FROM

Lesson 2.5: Built-in Tools (Edit replace_all)
Claude Code: Built-in Tools >

Q45 · D2 · 2.3 TOOL-DISTRIBUTION-CHOICE / TOOL-CHOICE · q-2-3-003 · Developer Productivity Tools CORRECT

A developer productivity platform uses an automated documentation agent. The agent must always extract metadata from source files before generating documentation, but in testing it sometimes skips the metadata extraction step and produces documentation from incomplete information. What is the correct tool_choice configuration to enforce the mandatory first step?

A Set tool_choice to 'auto' and add a system prompt instruction: 'You must call extract_metadata before any other tool.'

With 'auto', the model decides whether to call a tool at all. A system prompt instruction is advisory, not enforced — the model can still skip the step. This does not guarantee the mandatory first step.

D Remove all other tools except `extract_metadata` from the agent's configuration so it has no choice.

This prevents the agent from performing any other task after metadata extraction. The agent needs access to documentation generation tools for subsequent steps. Forced tool selection achieves the constraint without crippling the agent.

C Set `tool_choice` to forced selection with the specific tool name `extract_metadata` for the first turn, then switch to 'auto' for subsequent turns.

Forced selection `{'type': 'tool', 'name': 'extract_metadata'}` ensures the model must call this specific tool. The model cannot skip it or choose a different tool. After the first turn completes, switching to 'auto' allows normal operation for documentation generation.

B Set `tool_choice` to 'any' so the model must call a tool, relying on the tool description to guide it to `extract_metadata` first.

'any' forces the model to call a tool but lets it choose which one. It may select a different tool first, still skipping `extract_metadata`. This guarantees a tool call but not the correct tool call.

WHERE THIS COMES FROM

Lesson 2.3: Tool Distribution and Tool Choice (`tool_choice` configuration)

Anthropic: Tool Use Documentation ↗

Q46 · D3 · 3.6 CI/CD-INTEGRATION / P-FLAG · q-3-6-001 · CI/CD Pipeline Integration **CORRECT**

A CI pipeline script runs `'claude "Analyse this PR"'` but the job hangs indefinitely. Logs show Claude Code is waiting for interactive input. What is the correct fix?

D Add the `--batch` flag: `claude --batch "Analyse this PR"`

`--batch` is not a real Claude Code CLI flag. This option references a non-existent feature.

A Add the `-p` flag: `claude -p "Analyse this PR"`

The `-p` (`--print`) flag is the documented way to run Claude Code in non-interactive mode. It processes the prompt, outputs the result to `stdout`, and exits without waiting for user input.

B Set the environment variable `CLAUDE_HEADLESS=true` before running the command

`CLAUDE_HEADLESS` is not a real Claude Code environment variable. This option references a non-existent feature.

C Redirect `stdin` from `/dev/null`: `claude "Analyse this PR" < /dev/null`

Unix `stdin` redirection does not properly address Claude Code's interactive mode. The `-p` flag is the correct, documented approach.

WHERE THIS COMES FROM

Lesson 3.6: CI/CD Integration (-p flag)

Claude Code: Headless / CLI Reference ›

Q47 · D3 3.6 CI/CD INTEGRATION / SESSION-ISOLATION q-3-6-002 · CI/CD Pipeline Integration CORRECT

A CI/CD pipeline uses Claude Code to generate tests in one step and then review the same code in a subsequent step. The team notices that the review step rarely finds issues with the generated tests, but human reviewers consistently find problems. Both CI steps use the -p flag correctly. What is the most likely cause and fix?

C The review step runs the same model tier as generation, which is not capable enough to critique its own output; upgrade the review step to a more powerful model

Model capability is not the bottleneck. A more powerful model still inherits self-review bias when it runs in the generating session's context, because it retains the reasoning it used to justify the code. The pattern here (a review that passes work human reviewers reject) points to shared reasoning context, which is fixed by running the review as an independent session with no shared context, not by upgrading the model.

A The review step needs --output-format json to produce structured findings that can be parsed programmatically

The output format affects how findings are reported, not whether issues are detected. The problem is that the review misses issues entirely, not that findings are in the wrong format.

D The generation and review steps must run as independent sessions with no shared context, so the review instance approaches the code without the original reasoning bias

The same Claude session (or a session with carried-over context) that generated code is less effective at reviewing its own changes because it retains the reasoning context from generation. An independent review instance without that context is more likely to catch issues, matching what human reviewers find. Session context isolation for independent reviews is a key CI integration concept.

B The CLAUDE.md file lacks testing standards and review criteria, so the review has no project-specific context

While documenting testing standards in CLAUDE.md is important, it would affect the quality of both generation and review. The specific pattern here — review missing issues that humans catch — points to self-review bias, not missing context.

WHERE THIS COMES FROM

Lesson 3.6: CI/CD Integration (Session context isolation)

Q48 · D1 1.3 SUBAGENT-INVOCATION-CONTEXT / STRUCTURED-METADATA q-1-3-001 INCORRECT

A synthesis agent produces a report with several claims that have no source attribution. The web search and document analysis subagents are working correctly and returning well-sourced results. What is the most likely root cause?

D The synthesis agent needs access to the web search tool directly

Giving the synthesis agent web search tools violates the principle of scoped tool access and does not address the context passing failure.

A The synthesis agent's system prompt does not instruct it to include citations, so it drops source attribution even when given well-sourced input.

Even with a citation instruction, the synthesis agent cannot attribute claims to sources that were never passed to it. The lesson is explicit: do not blame the synthesis prompt; the root cause is the coordinator dropping structured metadata. *(your answer)*

C The web search subagent needs to return results in a different format

The web search subagent is working correctly. The issue is how the coordinator passes its results to the synthesis agent.

B The coordinator passes content to the synthesis agent stripped of structured metadata like source URLs and document names.

Context passing must include structured data that separates content from metadata. Without source URLs and document names, the synthesis agent has no attribution information to include. *(correct answer)*

WHERE THIS COMES FROM

Lesson 1.3: Subagent Invocation and Context Passing (Structured metadata format)

Lesson 1.3: Subagent Invocation and Context Passing (Context passing)

Q49 · D5 5.4 CODEBASE-EXPLORATION / SCRATCHPAD q-5-4-001

INCORRECT

A Claude Code agent is exploring an unfamiliar codebase. After investigating several modules, it starts referencing 'typical repository patterns' instead of the specific class names and dependency chains it discovered earlier. What is the most effective mitigation?

B Have the agent maintain scratchpad files recording key findings and reference them for subsequent questions

Scratchpad files persist knowledge across context boundaries, directly counteracting context degradation by keeping critical discoveries accessible. *(correct answer)*

C Restart the session and ask the agent to explore more efficiently

Restarting loses all accumulated knowledge without addressing the underlying context degradation problem.

A Increase the model's context window to accommodate more discovery output

Context degradation is not about running out of tokens — it is about the model losing grip on earlier findings as verbose output accumulates. *(your answer)*

D Pre-load the entire codebase structure into the initial context before exploration begins

This would consume context budget before exploration even starts and does not address degradation during the session.

WHERE THIS COMES FROM

Lesson 5.4: Codebase Exploration and Context Degradation (Context degradation)

Lesson 5.4: Codebase Exploration and Context Degradation (Scratchpad files)

Q50 · D2 · 2.1 TOOL-SCHEMA-DESIGN / SYSTEM-PROMPT · q-2-1-002 · Developer Productivity Tools **INCORRECT**

An agent has two tools: 'analyze_content' ('Analyses content') and 'extract_web_results' ('Extracts data from web pages'). After a system prompt update added 'Always analyse content before responding', the agent started routing web-extraction tasks to 'analyze_content' despite the unchanged descriptions. What is the most likely cause and fix?

B The tool descriptions need few-shot examples added to the system prompt showing when to use each tool.

Few-shot examples add token overhead and do not address the root cause. The system prompt instruction is directly triggering the wrong tool via keyword association, and adding more content to the system prompt may compound the issue. (your answer)

C The analyze_content tool should be renamed to something less generic, such as summarize_document, to avoid all future conflicts.

Renaming is a valid longer-term improvement but does not address the immediate cause: the system prompt instruction creating a keyword match. The prompt conflict would persist with any tool whose name overlaps with prompt wording.

D The tool descriptions have degraded over time and need to be rewritten with clearer boundaries between the two tools.

The question states tool descriptions were not changed and routing was correct before the system prompt update. The descriptions are not the root cause — the new system prompt instruction is.

A The system prompt's 'analyse content' phrasing keyword-matches the analyze_content tool; rephrase it to remove the overlap.

Keyword-sensitive instructions in system prompts can create unintended tool associations that override well-written descriptions. The phrase 'analyse content' directly matches the tool name, so the model favours it regardless of the task. The fix is to review and rephrase the system prompt after any update. (correct answer)

WHERE THIS COMES FROM

Lesson 2.1: Tool Interface Design (System prompt interactions)

Q51 · D1 · 1.5 AGENT-SDK-HOOKS / PRETOOLUSE-HOOKS · q-1-5-013 · Developer Productivity Tools **CORRECT**

A developer-tool agent writes code and runs shell commands. A system prompt instruction ('Do not run destructive commands or access files outside the project') is violated 3% of the time in testing. The team requires this rule to hold absolutely. What should the architect implement?

B PreToolUse hooks that scan shell commands for destructive patterns and validate file paths before execution.

PreToolUse hooks intercept tool calls before execution. A hook can pattern-match against destructive commands like 'rm -rf' and validate that file paths fall within the project directory, blocking any violation with 100% reliability. This is deterministic enforcement for a safety-critical requirement.

C Implement PostToolUse hooks that detect and roll back destructive commands after they execute

PostToolUse hooks run after execution. By the time a PostToolUse hook detects 'rm -rf', the files are already deleted. Destructive commands cannot be reliably rolled back. Prevention via PreToolUse is required, not post-execution detection.

A Strengthen the system prompt with more specific examples of forbidden commands and add few-shot demonstrations

The prompt already fails 3% of the time. Strengthening it may reduce the failure rate but cannot eliminate it. Destructive operations require 100% prevention, which only deterministic enforcement can guarantee.

D Remove the shell execution tool entirely to prevent any possibility of destructive commands

Removing shell execution eliminates the agent's ability to run any commands, including legitimate ones needed for its development workflow. The correct approach is surgical — block specific dangerous patterns while allowing legitimate usage.

WHERE THIS COMES FROM

Lesson 1.5: Agent SDK Hooks (PreToolUse hooks)

Anthropic: Agent SDK Hooks ↗

Q52 · D2 2.1 TOOL-SCHEMA-DESIGN / SPLITTING q-2-1-008 · Developer Productivity Tools INCORRECT

A developer productivity platform has a tool called analyze_content with the description 'Analyses content from various sources.' Logs show the agent uses it for web scraping, document parsing, and code analysis indiscriminately, leading to poor results for each use case. What is the most effective fix?

D Implement a routing layer that inspects the input content type and dispatches to the appropriate internal processing function within analyze_content.

An internal routing layer hides the content type distinction from the agent. Splitting into separate tools exposes the distinction through tool descriptions, enabling the LLM's natural language understanding to select the right tool.

A Add detailed few-shot examples to the system prompt showing which types of content the tool handles well and which it does not.

Few-shot examples add token overhead and treat the symptom. The root cause is that a single generic tool is trying to serve three distinct use cases, each requiring different processing logic. (your answer)

B Improve the tool description to list all supported content types and their expected formats.

A better description on a generic tool does not change the fact that the tool's processing logic is too broad. When a tool serves multiple distinct purposes, the fix is to split it into purpose-specific tools.

C Split analyze_content into purpose-specific tools: extract_web_results, parse_document, and analyze_code, each with a narrow remit.

Tool splitting replaces a generic tool with purpose-specific alternatives that each have a clear, narrow description and defined input/output contracts. Each resulting tool handles one use case well, which eliminates ambiguity about when to use it. (correct answer)

WHERE THIS COMES FROM

Lesson 2.1: Tool Interface Design (Tool splitting)

Lesson 2.1: Tool Interface Design (Tool descriptions)

Q53 · D1 1.5 AGENT-SDK-HOOKS / HOOKS-VS-PROMPTS q-1-5-014 · Developer Productivity Tools INCORRECT

A developer-tool agent has two guardrails: a PreToolUse hook blocking file writes outside the project directory (enforced 100% of the time), and a system prompt instruction 'Always create a backup before overwriting existing files' (followed 88% of the time). A senior engineer wants all guardrails converted to hooks for consistency. What is the correct assessment?

D The backup instruction should be converted to a hook because 88% is an unacceptable failure rate

Whether 88% is acceptable depends on the consequences of failure. Missing a backup in a version-controlled codebase is recoverable, so 88% compliance is reasonable for a prompt-based best practice. The 12% failure rate does not justify the implementation complexity of a hook for a non-critical guideline.

B The directory restriction is rightly a hook; the backup instruction can stay a prompt because a missed backup is recoverable.

The guardrail strategy should match the severity of the consequence. Writing outside the project directory could damage the host system — this demands deterministic enforcement via a hook. Missing a backup is inconvenient but recoverable (files are in version control). The 88% compliance rate from the prompt is acceptable for a non-critical best practice. (correct answer)

A The senior engineer is correct: every guardrail, including the recoverable backup rule, should be converted to a deterministic hook for maximum reliability.

Not all guardrails need deterministic enforcement. Converting everything to hooks adds implementation complexity for no benefit on recoverable rules. Match the enforcement mechanism to the severity of the requirement; a missed backup in version control is fine as a prompt. (your answer)

C Both guardrails should be prompt instructions to reduce implementation complexity

Prompt instructions for the directory restriction would have a non-zero failure rate for a safety-critical boundary. Writing outside the project directory could cause system damage. Safety-critical requirements must use deterministic hooks, not probabilistic prompts.

WHERE THIS COMES FROM

Lesson 1.5: Agent SDK Hooks (Hooks vs prompts)

Lesson 1.5: Agent SDK Hooks (Decision framework)

Q54 · D3 · 3.1 CLAUDE-MD-HIERARCHY / POSTTOOLUSE-ENFORCEMENT · q-3-1-014 · CI/CD Pipeline Integration · CORRECT

A fintech company requires that all API response payloads are normalised to snake_case before being logged, and that any file write to the src/api/ directory is automatically linted. They want these enforcements to be deterministic and not rely on the model remembering instructions. Which hook configuration achieves both requirements?

A A PreToolUse hook on file writes that runs the linter before the write completes, and a PostToolUse hook on API response handling that normalises field names to snake_case

PreToolUse fires before the tool runs, so there is no file content to lint yet. Linting must happen after the file is written (PostToolUse). The ordering of hooks to actions is incorrect here.

B A PostToolUse hook on file writes to src/api/ that runs the linter on the created file, and a PostToolUse hook on data processing that normalises response payloads to snake_case

Both are PostToolUse hooks because both act on completed outputs. The linter runs after a file write to src/api/ is complete, validating the written content. The normalisation hook processes API response data after it is retrieved, converting fields to snake_case before logging. Both provide deterministic enforcement independent of model instructions.

C Add both requirements to CLAUDE.md with clear examples, and add a PreToolUse hook that reminds the model about these rules before each tool call

CLAUDE.md instructions and PreToolUse reminders both rely on the model's compliance, which is not deterministic. The requirement explicitly states that enforcement must not depend on the model remembering instructions.

D A PreToolUse hook that blocks any file write to src/api/ unless the file has already been linted, and a PreToolUse hook that blocks API calls unless snake_case normalisation is configured

PreToolUse blocking assumes the file exists before the write, which is contradictory. You cannot lint a file that has not been written yet. PostToolUse hooks that process output after completion are the correct approach for validation and normalisation.

WHERE THIS COMES FROM

Claude Code: Hooks ›

Q55 · D1 · 1.2 ORCHESTRATION-PATTERNS / CONTEXT-PASSING · q-1-2-007 · INCORRECT

A team runs a coordinator agent that delegates refactoring work to three specialist subagents: a schema-migration agent, an API-layer agent, and a test-update agent. The test-update agent frequently produces

tests that reference database schemas the schema-migration agent has already renamed. The subagents do not communicate with each other. What is the root cause?

C

The coordinator is not passing the schema-migration agent's output (including renamed schemas) as context when delegating to the test-update agent

In hub-and-spoke orchestration, the coordinator manages all inter-agent communication. Subagents do not inherit each other's context. The coordinator must explicitly pass relevant outputs from earlier subagents as context to downstream subagents. Here, the renamed schema mappings must be injected into the test-update agent's context. (correct answer)

B

The test-update agent should have read access to the schema migration files so it can discover the new names itself

While this might work as a workaround, it misses the architectural root cause. The coordinator is responsible for passing all necessary context to each subagent. The test-update agent should not need to discover information that the coordinator already has.

A

The subagents need a shared message bus so the test-update agent can query the schema-migration agent for the latest schema names

In a hub-and-spoke multi-agent architecture, subagents must never communicate directly. All communication flows through the coordinator. Adding a message bus violates this principle and creates coordination complexity.

D

The schema-migration agent and test-update agent should run sequentially rather than in parallel to avoid race conditions

Sequencing alone does not solve the problem if the coordinator does not pass the schema-migration agent's results to the test-update agent. The issue is missing context injection, not execution order. (your answer)

WHERE THIS COMES FROM

Lesson 1.2: Multi-Agent Orchestration (Isolation principle)

Lesson 1.3: Subagent Invocation and Context Passing (Context passing)

Q56 · D1 1.2 ORCHESTRATION-PATTERNS / ISOLATION q-1-2-010 · Content Moderation and Classification System CORRECT

The moderation coordinator dispatches posts to specialist subagents. The text classifier has been extended to also handle image analysis: its prompt is now 2,500 tokens, its tool list has grown to 12 tools, and accuracy has dropped on both text and image tasks. What principle was violated and what is the fix?

D

Split the combined agent into three narrower specialists: text sentiment, text policy, and image analysis

Over-splitting into too many narrow specialists adds coordination overhead without clear benefit. The original architecture had a text classifier and an image analyser, which is the correct level of decomposition. Restoring that structure is the fix.

A

The text classifier's context window is too small — upgrade to a model with a larger context window to handle the expanded prompt

Context window size is not the issue. Combining unrelated responsibilities in a single agent degrades performance because the model must juggle competing instructions and tools. A larger context window does not fix scope creep.

B Isolation was violated; restore the text classifier to text-only work with scoped tools and keep image analysis separate.

Each subagent should have a focused responsibility and a scoped tool set (4-5 tools maximum). Combining text and image analysis in one agent violates isolation, bloats the prompt, and confuses the model. Restoring separate specialists with scoped tools improves accuracy for both tasks.

C Add better instructions to the combined subagent's prompt to clarify when it should use text tools versus image tools

Better instructions do not address the fundamental problem of an overloaded agent. Twelve tools and 2,500 tokens of instructions degrade performance regardless of instruction quality. The fix is architectural, not instructional.

WHERE THIS COMES FROM

Lesson 1.2: Multi-Agent Orchestration (Isolation principle)

Lesson 1.4: Workflow Enforcement and Handoff (Scoped tool access)

Q57 · D1 1.3 SUBAGENT-INVOCATION-CONTEXT / SCOPED-TOOLS q-1-3-013

CORRECT

An architect is designing a customer support system using the Claude Agent SDK. The system needs a coordinator agent that can delegate to a billing specialist and a technical support specialist. Each specialist needs different tools. How should the architect configure the AgentDefinitions?

D Give the coordinator all tools and have it handle all requests directly, eliminating the need for subagents

A single agent with all tools has a higher error rate due to tool selection confusion. Specialist subagents with scoped tools produce more reliable results. As a rough guide, aim for a small set of around 4-5 tools per agent.

B Create separate AgentDefinitions for each specialist with scoped tools (4-5 tools each), and give the coordinator the Task tool to spawn them

Each specialist gets its own AgentDefinition with its tools restricted to the 4-5 tools relevant to its role. The coordinator has the Task tool (renamed to Agent in current Claude Code v2.1.63; 'Task' still works as a backward-compatible alias) in its allowedTools so it can spawn specialists. This enforces separation of concerns and scoped tool access.

A Create one AgentDefinition with all tools from both specialists, and use prompt instructions to tell the agent which tools to use in each context

Combining all tools into one agent violates the principle of scoped tool access. With too many tools, the agent is more likely to use the wrong tool. Each specialist should have only the 4-5 tools relevant to its role.

C Create the specialists as separate API endpoints and have the coordinator call them via HTTP rather than the Task tool

Using HTTP calls instead of the Task tool bypasses the Agent SDK's built-in orchestration capabilities. The Task tool is the designed mechanism for subagent invocation and provides proper context management.

WHERE THIS COMES FROM

- Lesson 1.4: Workflow Enforcement and Handoff (Enforcement spectrum: scoped tools)
- Lesson 1.3: Subagent Invocation and Context Passing (Task tool)
- Anthropic: Claude Agent SDK Overview ↗

Q58 · D3 · 3.1 CLAUDE-MD-HIERARCHY / HIERARCHY · q-3-1-005 · Developer Productivity Tools CORRECT

A consultancy works across 12 client projects simultaneously. Each developer has personal preferences (editor keybindings, alias shortcuts) and the consultancy has firm-wide coding standards. Each client project has its own specific conventions. Some client projects have subsystems with additional specialised rules. What is the correct configuration architecture?

B User ~/.claude/CLAUDE.md for personal preferences and firm-wide standards; project .claude/CLAUDE.md for client-specific conventions; directory-level CLAUDE.md or .claude/rules/ with paths for subsystem-specific rules

User-level ~/.claude/CLAUDE.md applies only to the individual developer and is not shared via version control. Placing firm-wide standards there means a colleague who clones a client project does not receive them, which is the classic configuration-hierarchy bug. Shared standards belong at the project level.

A User ~/.claude/CLAUDE.md for personal preferences; project .claude/CLAUDE.md for firm-wide standards and client conventions; directory-level CLAUDE.md or .claude/rules/ with paths for subsystem rules

Personal preferences are per-developer, so they live at user level, which is not shared via version control. Firm-wide standards and client conventions must reach everyone who works on a project, so they belong at the project level, which is committed to the repository and distributed on clone; firm-wide standards can be kept in a shared file and pulled into each project's CLAUDE.md with an @import to stay modular. Putting shared standards at the user level is the classic hierarchy bug, where a colleague who clones a project would not receive them. Subsystem rules belong at the directory level or in path-scoped .claude/rules/.

C A single CLAUDE.md at the root of each project containing all four levels of configuration, with clear section headings

Combining all configuration levels into a single file defeats the purpose of the hierarchy. Personal preferences would be version-controlled and shared with the team, firm-wide standards would need duplication across 12 projects, and the file would be bloated with subsystem rules that only apply to specific directories.

D User ~/.claude/CLAUDE.md for everything; use @ path imports to pull in project-specific files from each repository

User-level CLAUDE.md loads for every session across all projects. Importing 12 client project configurations into the user-level file would load all client conventions for every project, wasting tokens and causing conflicting instructions.

WHERE THIS COMES FROM

- Lesson 3.1: CLAUDE.md Hierarchy and Scoping (Three-level hierarchy)
- Lesson 3.1: CLAUDE.md Hierarchy and Scoping (.claude/rules/)

Q59 · D2 · 2.5 BUILT-IN-TOOLS / EDIT-RECOVERY · q-2-5-007 · Developer Productivity Tools INCORRECT

A developer asks Claude Code to modify a function in a large file. The Edit tool fails with a 'non-unique match' error because the target text appears in multiple locations within the file. What is the correct recovery approach?

B

Use Read to load the full file contents, then use Write to output the complete modified file with the change applied.

Rewriting the whole file with Write is wasteful and unsafe — it loses the surgical-edit guarantees of Edit and risks dropping content. This is a last resort when no unique context exists, not the designed recovery path.

D

Split the file into smaller files so that each occurrence appears in only one file, then use Edit on each.

Restructuring files to accommodate tool limitations is disproportionate effort. The Edit tool's own recovery path (more context or replace_all) handles non-unique matches without changing the codebase.

C

Expand the Edit search string to include more surrounding context until it becomes unique in the file.

The Edit tool's documented recovery path: 'Either provide a larger string with more surrounding context to make it unique, or use replace_all to change every instance of old_string.' Expanding context preserves Edit's surgical safety. If every occurrence should be changed identically, replace_all is the alternative. (correct answer)

A

Switch to Bash with sed to perform the replacement using line numbers instead of text matching.

Bash with sed bypasses the built-in Edit tool's safety guarantees and is fragile when line numbers shift. The Edit tool documents the correct first recovery step (expand context or use replace_all). (your answer)

WHERE THIS COMES FROM

Lesson 2.5: Built-in Tools (Edit non-unique match recovery)

Claude Code: Built-in Tools ›

Q60 · D3 · 3.3 PATH-SPECIFIC-RULES / MULTI-RULESET · q-3-3-002 · CI/CD Pipeline Integration CORRECT

A DevOps team's polyglot codebase has Terraform files in terraform/, Kubernetes manifests in k8s/, and Dockerfiles scattered throughout various service directories. They want Claude Code to automatically apply infrastructure-specific conventions when editing each type of file, without loading all conventions for every session. What is the correct approach?

B

Add all infrastructure conventions to the root CLAUDE.md with clear section headings so the model knows which section applies

Root CLAUDE.md loads for every session. All infrastructure conventions would consume tokens even when editing TypeScript application code. Section headings do not prevent the conventions from being loaded — the model still processes the entire file.

C Create a single `.claude/rules/infrastructure.md` file with paths: `["**/*"]` containing all infrastructure conventions

The glob pattern `**/*` matches every file in the codebase, which means the infrastructure conventions would load for every session — exactly the same as putting them in root `CLAUDE.md`. This defeats the purpose of conditional loading.

A Create three directory-level `CLAUDE.md` files: one in `terraform/`, one in `k8s/`, and one in the project root for Docker conventions

Directory-level `CLAUDE.md` files in `terraform/` and `k8s/` would work for those specific directories, but Dockerfiles are scattered throughout various service directories. A single root `CLAUDE.md` for Docker conventions would load for every session, defeating the token efficiency goal. This approach does not handle the cross-directory Dockerfile pattern.

D Create three path-scoped `.claude/rules/` files (`terraform.md`, `kubernetes.md`, `docker.md`), each targeting its own file types

Each rule file targets specific infrastructure file types with appropriate glob patterns. Terraform conventions load only when editing Terraform files, Kubernetes conventions only for k8s manifests, and Docker conventions only for Dockerfiles — regardless of which directory the Dockerfiles are in. This provides token-efficient conditional loading for all three file types.

WHERE THIS COMES FROM

Lesson 3.3: Path-Specific Rules (Multiple rule files)

Lesson 3.3: Path-Specific Rules (Glob patterns)

SUPPORT

Found this useful?

The site is free and always will be. If the mock exam helped, you can buy me a coffee. Support goes to keeping the lights on.

= BUY ME A COFFEE